Two pointers. Căutare binară pe rezultat

Infogym • grupa 7–8 avansați • 24 octombrie 2026

Andrei Mihai Ciobanu





Hai la Olimpiadă!

Cuprins

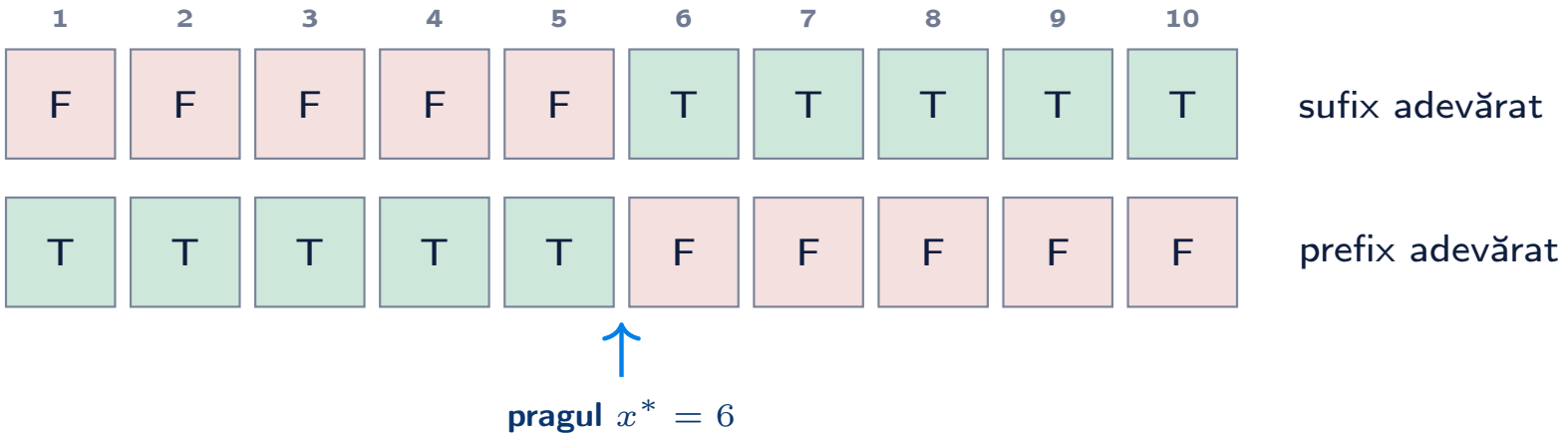
1. Monotonia
2. Two pointers
3. Căutare binară pe rezultat
4. Alte variante
5. Recapitulare și probleme propuse

Monotonia

Definiție

O proprietate $P(x)$ este **monotonă** dacă își schimbă valoarea cel mult o dată când x crește. De exemplu, odată adevărată, rămâne adevărată pentru toate valorile mai mari:

$$P(x_1) \text{ adevărată și } x_1 \leq x_2 \implies P(x_2) \text{ adevărată}$$



- Definiția este valabilă și invers: $\mathrm{T} \ldots \mathrm{T}\,\mathrm{F} \ldots \mathrm{F}$ (prefix adevărat, sufix fals), unde odată falsă rămâne falsă
- Problema se reduce la găsirea **pragului** x^*, fără a verifica toate valorile

Exemple de proprietăți monotone

Situație	**Proprietate**	**Variabila**
Tablou sortat crescător	$a[i] \geq v$	poziția i
Elemente nenegative	suma $a_l + \cdots + a_r$ este $\geq S$	capătul r
Fabrică cu mai multe utilaje	„în T secunde se produc cel puțin t produse"	timpul T
Transport în k drumuri	„toate obiectele încap cu capacitatea C"	capacitatea C
Perechi $a_i + b_j$	„există cel puțin k perechi cu suma $\leq x$"	valoarea x

Două moduri de a exploata monotonia

	Căutare binară pe rezultat	**Two pointers**
Unde apare monotonia	predicatul $\mathrm{check}(x)$ pe domeniul răspunsurilor	pentru fiecare l, cel mai mare r valid, $r(l)$, nu scade când l crește
Ce se elimină	jumătate din domeniu la fiecare test	pozițiile la care indicii nu se mai întorc
Cost	$O(\log V)$ teste $\times$ costul verificării	$O(n)$ amortizat

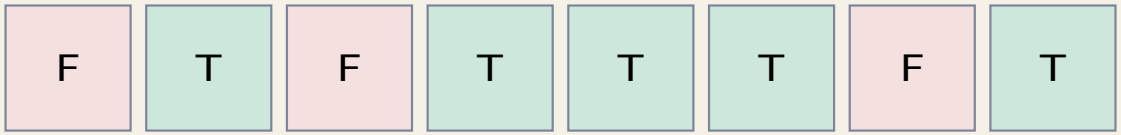
Aceeași idee în ambele cazuri: o proprietate monotonă permite ca **nu toate cazurile** să fie verificate.

Când monotonia lipsește

Two pointers cu numere negative

Cea mai scurtă secvență cu suma $\geq S$ în șirul $[5, -10, 6]$ pentru $S = 6$ este $[6]$, de lungime 1.
Sumele secvențelor care încep în $l = 1$ sunt $5, -5, 1$ și nu ajung niciodată la 6, dar secvența $[6]$ începe în $l = 3$. Suma **nu mai este monotonă** în r, deci extinderea lui r nu se poate opri la prima sumă mică, iar $r(l)$ nu mai este monoton în l.

Căutare binară cu predicat care nu este monoton



Testul din mijloc nu mai spune în ce jumătate se află răspunsul: soluția poate fi eliminată din greșeală.

Cuprins

1 Monotonia

2 Two pointers

3 Căutare binară pe rezultat

4 Alte variante

5 Recapitulare și probleme propuse

Two pointers: ideea

- Doi indici, l și r, parcurg tabloul; fiecare se deplasează **într-un singur sens**, fără întoarceri
- Două variante frecvente:
  - **fereastră glisantă**: $l \leq r$, ambii cresc; se lucrează cu secvențe $[l, r]$
  - **capete opuse**: l crește, r scade; se lucrează de obicei pe tablouri sortate după o anumită regulă
- **"Rezumatul" ferestrei**: despre elementele din $[l, r]$ se păstrează anumite informații (suma, frecvențele, numărul de valori distincte) care se actualizează rapid (în $O(1)$ sau $o(\log n)$) când un element intră (r crește) sau iese (l crește)

Monotonia din spatele tehnicii

Pentru fiecare l, cel mai mare r cu $[l, r]$ validă, notat $r(l)$, **nu scade** când l crește: l avansează și r se ajustează în consecință.

Fereastră glisantă: cea mai lungă secvență cu suma $\leq S$

Enunț

Se dă șirul $a = (2, 1, 3, 1, 1, 4, 2)$ de numere nenegative (indexat de la 1) și $S = 6$. Să se determine lungimea celei mai lungi secvențe cu suma cel mult S. Pentru fiecare l, capătul r se extinde cât timp suma rămâne $\leq S$.

1	2	3	4	5	6	7	
2	1	3	1	1	4	2	$l = 1$: $r = 3$, **sumă** 6, **lungime** 3
2	1	3	1	1	4	2	$l = 2$: $r = 5$, **sumă** 6, **lungime** 4 $\leftarrow$ **maximul**
2	1	3	1	1	4	2	$l = 3$: $r = 5$, **sumă** 5, **lungime** 3
2	1	3	1	1	4	2	$l = 4$: $r = 6$, **sumă** 6, **lungime** 3
2	1	3	1	1	4	2	$l = 5$: $r = 6$, **sumă** 5, **lungime** 2
2	1	3	1	1	4	2	$l = 6$: $r = 7$, **sumă** 6, **lungime** 2
2	1	3	1	1	4	2	$l = 7$: $r = 7$, **sumă** 2, **lungime** 1

Șablon two pointers: secvențe cu suma $\leq S$

```
// Numarul de secvente cu suma <= S (tabloul a este indexat de la 1)
long long answer = 0, sum = 0;
for (int l = 1, r = 0; l <= n; ++l) {
    // extindem cat putem
    while (r + 1 <= n && sum + a[r + 1] <= S) {
        // adaugam a[r + 1] in secventa
        sum += a[++r];
    }
    // secventele valide care incep la l au capatul drept pana in r
    answer += r - l + 1; // secventa poate fi vida: atunci se aduna 0
    // eliminam a[l]
    if (r >= l) sum -= a[l]; else r = l; // fereastra vida: r nu ramane in urma lui l
}
```

- **Complexitate** $O(n)$: l și r doar cresc și fiecare face cel mult n pași, deci în total cel mult $2n$ mutări
- Funcționează deoarece elementele sunt **nenegative**: scoaterea lui $a[l]$ nu poate crește suma, iar adăugarea lui $a[r]$ nu o poate scădea
- Pentru **cea mai lungă** secvență, în locul adunării se folosește `best = max(best, r - l + 1)`; răspunsul poate ajunge la ordinul n^2, deci `long long`

Șablon two pointers: secvențe cu suma $\geq S$

```
// Numarul de secvente cu suma >= S (S > 0, elemente nenegative)
// Alternativ: numarul total de secvente minus cele cu suma <= S - 1
long long answer = 0, sum = 0;
for (int l = 1, r = 0; l <= n; ++l) {
    // extindem cat timp conditia nu e satisfacuta
    while (r + 1 <= n && sum < S) {
        sum += a[++r];
    }
    // atentie: secventa s-ar putea sa nu fie valida!
    if (sum >= S) {
        // secventele valide care incep la l au capatul drept de la r pana la n
        answer += n - r + 1;
    }
    // eliminam a[l]
    sum -= a[l];
}
```

- Aici fereastra se extinde **cât timp proprietatea nu este satisfăcută**; pentru fiecare l, primul r valid dă toate capetele $r, r+1, \ldots, n$
- Pentru cea mai **scurtă** secvență cu suma $\geq S$: `best = min(best, r - l + 1)` în locul adunării

Șablon two pointers general: proprietate oarecare

```
// Numarul de secvente cu o proprietate care se pastreaza cand secventa se micsoreaza
      .
// Uneori nu e usor de verificat daca secventa + elementul urmator e valida:
// il adaugam, iar daca strica proprietatea, il scoatem.
long long answer = 0;
for (int l = 1, r = 0; l <= n; ++l) {
    // daca proprietatea e satisfacuta si mai avem un element, extindem
    while (r + 1 <= n && ok()) {
        insert(++r);              // adauga a[r] in structura ferestrei
    }
    // s-ar putea sa fi extins cu un element prea mult
    if (!ok()) {
        remove(r--);              // scoate a[r] din structura ferestrei
    }
    if (ok()) {
        answer += r - l + 1;
    }
    // eliminam a[l]
    remove(l);
}
```

- Codul nu trebuie să fie întotdeauna atât de rigid, dar la probleme complicate se iau în calcul detaliile: secvențele vide, secvențele care nu mai pot fi extinse și totuși nu satisfac proprietatea
- O problemă bună pentru a testa înțelegerea tehnicii: **Mapat** (ONI 2026 cl. VII)

Capete opuse: Sum of Two Values (CSES 1640)

Enunț

Se dau n numere și un număr x. Să se găsească două poziții **distincte** $i \neq j$ cu $a_i + a_j = x$ sau să se afișeze `IMPOSSIBLE`.

- Se sortează perechile (valoare, poziție); $l = 0$, $r = n - 1$
- $a_l + a_r < x$: a_l nu se potrivește cu niciun element rămas (toate sunt $\leq a_r$), deci l++
- $a_l + a_r > x$: a_r nu se potrivește cu niciun element rămas (toate sunt $\geq a_l$), deci r−

l	r	$a_l + a_r$	decizie ($x = 13$, șir sortat 1 3 4 6 9 11)
0	5	$1 + 11 = 12 < 13$	$l \to 1$
1	5	$3 + 11 = 14 > 13$	$r \to 4$
1	4	$3 + 9 = 12 < 13$	$l \to 2$
2	4	$4 + 9 = 13$	găsit

Capete opuse: implementare

```
vector<pair<long long, int>> v(n);      // (valoare, pozitie)
for (int i = 0; i < n; i++) {
    cin >> v[i].first;
    v[i].second = i + 1;
}
sort(v.begin(), v.end());

int l = 0, r = n - 1;
while (l < r) {
    long long s = v[l].first + v[r].first;
    if (s == x) {
        cout << v[l].second << " " << v[r].second;
        return 0;
    }
    if (s < x) {
        l++;                // suma prea mica
    }
    else {
        r--;                // suma prea mare
    }
}
cout << "IMPOSSIBLE";
```

- Complexitate: $O(n \log n)$ pentru sortare, apoi $O(n)$. Sortarea strică pozițiile, de aceea se păstrează perechea (valoare, poziție); suma se ține în `long long`

Problemă rezolvată: Palindrom (Kilonova, OJI 2023 cl. VII)

Enunț

Un număr este palindrom dacă se citește la fel în ambele sensuri; poate fi făcut palindrom adăugând cifre la dreapta. Se dau n numere naturale (cel mult 50 de cifre). **Cerința 1:** numărul minim de cifre adăugate în total pentru ca toate numerele să devină palindrom. **Cerința 2:** cu cel mult S cifre adăugate în total, numărul maxim de termeni *consecutivi* care devin palindrom. **Restricții:** $n \leq 5 \cdot 10^4$, $0 \leq S \leq 5 \cdot 10^5$.

Exemplu

$12232,\ 131,\ 12345,\ 0,\ 7717 \Longrightarrow$ cerința 1: $2+0+4+0+1=7$. Pentru cerința 2, cu $S=4$ și încă două numere $(1244,\ 215809)$, răspunsul este 3: de exemplu $131, 12345, 0$ cu costul $0+4+0$.

Palindrom: rezolvare

- $\text{cost}(x)$ = lungimea lui x minus lungimea celui mai lung **sufix palindrom**: la 12232, sufixul 232 este palindrom, deci se adaugă 2 cifre ($12232\,21$)
- Cerința 1: suma costurilor. Cerința 2: cea mai lungă secvență de termeni consecutivi cu **suma costurilor** $\leq S$; costurile sunt nenegative $\Rightarrow$ fereastră glisantă
- Complexitate $O(n \cdot 50^2)$. Atenție la **fereastra vidă**: dacă un cost depășește S, r nu avansează, iar costul nu se scade din sumă

Problemă rezolvată: criptografie (Kilonova, ONI 2019 cl. VIII)

Enunț

Se dă un șir de n litere mici și un număr k. **Cerința 1:** să se numere secvențele (litere consecutive) în care fiecare literă apare de **cel mult k ori**; două secvențe sunt diferite dacă diferă începutul sau sfârșitul. **Cerința 2** ($k = 1$): cea mai lungă secvență cu litere distincte; la lungime egală, prima în ordine lexicografică. **Restricții:** $n \leq 10^5$.

Exemple

$k = 1$, `abac` $\Longrightarrow 8$: `a, ab, b, ba, bac, a, ac, c`.
Cerința 2: `abacba` $\Longrightarrow$ `acb` (secvențele `bac`, `acb`, `cba` au lungime maximă 3).

- Proprietatea este **ereditară**: o subsecvență a unei secvențe valide este validă $\Rightarrow$ fereastră glisantă, cu șablonul general
- Se păstrează `cnt[26]` și `bad` = numărul de litere care apar de mai mult de k ori; `ok()` $\Longleftrightarrow$ `bad == 0`. Cerința 1: $\sum_l (r - l + 1)$
- Cerința 2: aceeași fereastră cu $k = 1$; se păstrează cea mai lungă secvență, iar la lungime egală se compară lexicografic (lungimea este cel mult 26)
- Fiecare literă intră și iese o singură dată din fereastră $\Rightarrow O(n)$; răspunsul la cerința 1 este de ordinul n^2: `long long`

Cuprins

1. Monotonia
2. Two pointers
3. Căutare binară pe rezultat
4. Alte variante
5. Recapitulare și probleme propuse

Căutare binară pe rezultat: ideea

- Căutarea binară clasică caută o **poziție** într-un tablou sortat
- Căutarea binară **pe rezultat** caută direct **valoarea răspunsului**, fără ca tabloul să existe

Schema în trei pași

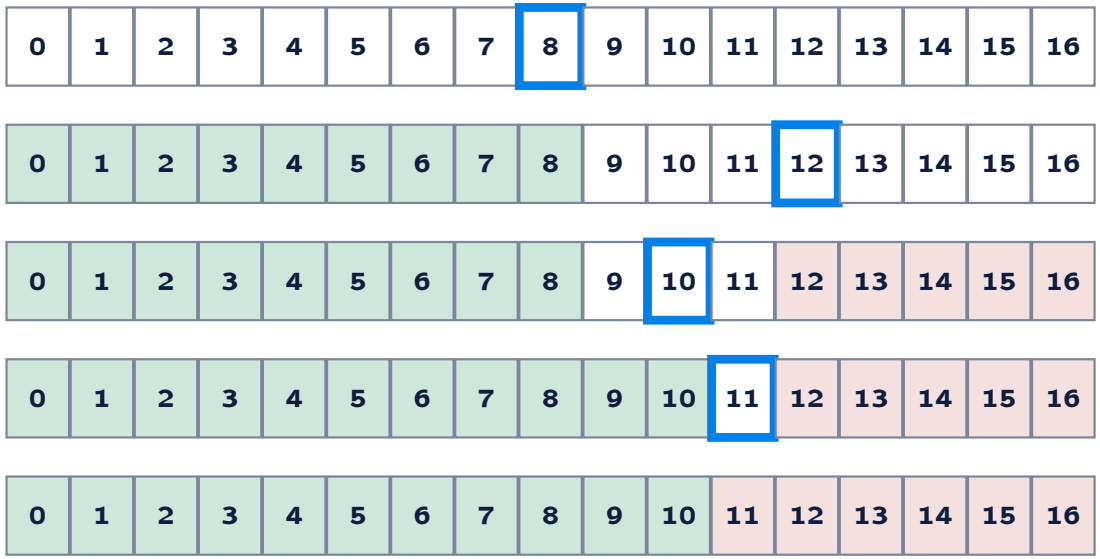
1. Se stabilește intervalul $[st, dr]$ în care se află răspunsul
2. Se scrie o funcție $\mathrm{check}(x)$: „se poate rezolva problema cu valoarea x?"
3. Se verifică faptul că check este **monotonă**, apoi se caută pragul: cel mai mare x cu $\mathrm{check}(x)$ adevărat (sau cel mai mic, în varianta simetrică)

- Cuvinte-cheie în enunț: „cel mai mic / cel mai mare x astfel încât...", „minimizează maximul", „maximizează minimul"

▷ *Știați că? Un număr din* $[1, 10^9]$ *se ghicește în cel mult* 30 *de întrebări „mai mare sau mai mic?", deoarece* $2^{30} > 10^9$. *Comanda* `git bisect` *aplică aceeași idee pe istoricul commit-urilor: „bun, . . . , bun, defect, . . . , defect" este un predicat monoton.*

Căutare binară: desfășurare

Se caută cel mai mare x cu $\text{check}(x)$ adevărat; aici $\text{check}(x)$ este adevărat exact pentru $x \leq 10$. Intervalul necercetat $[st, dr]$ începe cu $[0, 16]$; celula cu margine albastră este mid. Dacă $\text{check}(mid)$ este adevărat, $ans = mid$ și $st = mid + 1$; altfel $dr = mid - 1$.



■ adevărat (cunoscut) ■ fals (cunoscut) □ necunoscut

Intervalul se înjumătățește la fiecare test: aici 4 teste pentru 17 valori; pentru un domeniu de 10^{18} valori, circa 60 de teste.

Șablon

```
// cel mai mare x pentru care check(x) este adevarat
int st = ..., dr = ..., ans = ...;
while (st <= dr) {
    int mid = (st + dr) / 2;
    if (check(mid)) {
        ans = mid;
        st = mid + 1;
    }
    else {
        dr = mid - 1;
    }
}
```

```
// varianta pentru cel mai mic x pentru care check(x) este adevarat
    if (check(mid)) {
        ans = mid;
        dr = mid - 1;
    }
    else {
        st = mid + 1;
    }
```

- ans păstrează ultima valoare pentru care check a fost adevărat; $[st, dr]$ este intervalul încă necercetat
- `lower_bound(v)`, primul i cu $a[i] \geq v$, este varianta pentru minim cu $\text{check}(i) = (a[i] \geq v)$; `upper_bound(v)` folosește $a[i] > v$
- Complexitate: $O(\log V \cdot \text{cost}(\text{check}))$, unde $V = dr - st$

Detalii care contează

- **Limitele**: $[st, dr]$ trebuie să conțină răspunsul; se aleg din restricții (de exemplu $st = 1$, $dr = t \cdot \min k_i$ la Factory Machines). Dacă răspunsul poate lipsi, `ans` pornește de la o valoare care înseamnă „nu există" (de exemplu -1)
- **Overflow**: `(st + dr) / 2` depășește `int` când $st + dr > 2^{31}$; se folosește `long long` sau `st + (dr - st) / 2`. În `check` sumele pot depăși limitele, deci se oprește verificarea de îndată ce răspunsul este clar
- **Terminarea**: după fiecare test, mid iese din interval ($st = mid + 1$ sau $dr = mid - 1$), deci intervalul se micșorează strict; căutarea se oprește când $st > dr$
- **Numere reale**: condiția $st \leq dr$ nu mai are sens; se face un număr fix de iterații (de exemplu 100) cu $st = mid$ sau $dr = mid$; este metoda bisecției, cunoscută din analiza matematică
- **Verificarea trebuie să fie rapidă**: de obicei $O(n)$, adesea cu un algoritm greedy; dacă este costisitoare și sunt multe întrebări, se folosește căutarea binară în paralel

▷ *Știați că? În 2006, Joshua Bloch a arătat că `(low + high) / 2` depășește tipul `int` pentru tablouri foarte mari; eroarea a rămas nedescoperită aproape un deceniu în biblioteca standard Java și în cărți de referință.*

Problemă rezolvată: buldo (Kilonova, OJI 2020 cl. IX)

Enunț

Un teren are N zone consecutive, de înălțimi $h_1, \ldots, h_N$. Un buldozer alege o înălțime H pentru lamă, pornește fără pământ pe lamă ($C = 0$) și parcurge zonele de la stânga la dreapta: o zonă mai înaltă decât H este tăiată până la H (pământul ajunge pe lamă), una mai joasă este umplută cu pământ de pe lamă, care trebuie să ajungă. Să se afle H **maxim** cu care terenul poate fi nivelat. **Restricții:** $1 \leq N \leq 10^5$, $1 \leq h_i \leq 10^9$.

Exemplu

$N = 4$, $h = (5, 2, 1, 6) \Longrightarrow 2$. La $H = 2$ pământul de pe lamă este $3, 3, 2, 6$. La $H = 3$ este $2, 1$, iar zona 1 cere 2 unități: nu ajung.

- $\text{check}(H)$: după fiecare zonă, $C \leftarrow C + (h_i - H)$ și trebuie ca $C \geq 0$; se simulează în $O(N)$. Este **monotonă**: cu un H mai mic se taie mai mult și se umple mai puțin
- Se caută cel mai mare H cu $\text{check}(H)$ adevărat: șablonul pentru maxim, cu $st = 1$ (mereu posibil, deoarece $h_i \geq 1$) și $dr = 10^9$. Complexitate $O(N \log 10^9)$; C poate ajunge la 10^{14}: `long long`
- Condiția $C \geq 0$ înseamnă $h_1 + \cdots + h_i \geq i \cdot H$ pentru orice i, deci există și o soluție în $O(N)$: $H = \min_i \lfloor (h_1 + \cdots + h_i)/i \rfloor$

Problemă rezolvată: pietricele (Kilonova, ONI 2023 cl. IX)

Enunț

Se dau n pietre în șir, fiecare de un tip (literă mică), iar fiecare tip are o valoare. Pietrele se împart pe k rafturi, fiecare raft fiind o subsecvență nevidă de pietre consecutive; valoarea unui raft este suma valorilor pietrelor. **Cerința 1:** se maximizează cea mai mare valoare dintre rafturi. **Cerința 2:** se maximizează cea mai mică valoare dintre rafturi. **Restricții:** $1 \leq k \leq n \leq 2 \cdot 10^5$, valori în $[1, 10^9]$ (`long long`).

Exemplu (cerința 2)

`aabaacabcbaa`, $k = 3$, valorile $a = 1$, $b = 2$, $c = 3 \Longrightarrow 6$. Rafturile `aabaa` (6), `cab` (6), `cbaa` (7).

- Cerința 2 este „maximizează minimul”. $\mathrm{check}(x)$: „se pot forma cel puțin k rafturi de valoare $\geq x$?”. Greedy: raftul se închide imediat ce suma atinge x, iar restul se alipește ultimului raft. Monotonă: dacă x este posibil, și $x-1$ este $\Rightarrow$ șablonul pentru maxim, cu $st = 1$ și $dr =$ suma tuturor valorilor
- Cerința 1 nu cere căutare binară: raftul cel mai mare este maxim când celelalte $k-1$ rafturi au câte o piatră, deci răspunsul este cea mai mare sumă a unei secvențe de lungime $n-k+1$ (sume parțiale)
- Complexitate $O(n \log \sum v)$; sumele ajung la $2 \cdot 10^{14}$, deci `long long`

Varianta pentru minim: Factory Machines (CSES 1620)

Enunț

O fabrică are n utilaje care lucrează simultan; utilajul i produce un produs în k_i secunde. Care este timpul minim în care se pot produce t produse? **Restricții:** $1 \leq n \leq 2 \cdot 10^5$, $1 \leq t, k_i \leq 10^9$.

Exemplu

$n = 3$, $t = 7$, $k = (3, 2, 5) \implies 8$
În 8 secunde: $\lfloor 8/3 \rfloor + \lfloor 8/2 \rfloor + \lfloor 8/5 \rfloor = 2 + 4 + 1 = 7$; în 7 secunde: $2 + 3 + 1 = 6 < 7$.

- Răspunsul este un timp T **necunoscut**; $\text{check}(T)$: „se pot produce cel puțin t produse în T secunde?", adică $\sum_i \left\lfloor \frac{T}{k_i} \right\rfloor \geq t$. Este **monotonă**: cu mai mult timp nu se produc mai puține produse; este adevărată de la un prag în sus $\Rightarrow$ șablonul pentru minim
- Limite: $st = 1$ (deoarece $t \geq 1$), $dr = t \cdot \min k_i$ (valid: utilajul cel mai rapid produce singur t produse)

Factory Machines: implementare — $O(n \log(t \cdot \min k_i))$

```
bool check(long long T) {                  // se pot face t produse in T secunde?
    long long cnt = 0;
    for (long long ki : k) {
        cnt += T / ki;
        if (cnt >= t) return true;         // oprire anticipata: evita overflow
    }
    return false;
}

long long st = 1, dr = t * *min_element(k.begin(), k.end()), ans = dr;
while (st <= dr) {
    long long mid = st + (dr - st) / 2;
    if (check(mid)) {
        ans = mid;
        dr = mid - 1;
    }
    else {
        st = mid + 1;
    }
}
cout << ans;
```

Overflow

dr poate ajunge la 10^{18}, iar suma din check la $2 \cdot 10^{23}$: suma se oprește imediat ce atinge t. Complexitatea este de circa $2 \cdot 10^5 \cdot 60$ operații.

Cuprins

1. Monotonia
2. Two pointers
3. Căutare binară pe rezultat
4. Alte variante
5. Recapitulare și probleme propuse

Căutare binară prin puteri ale lui 2 și căutare exponențială

Binary lifting

- În loc de intervalul $[st, dr]$ se ține poziția curentă x și se testează $x + 2^p$ pentru p descrescător; dacă predicatul este încă adevărat, x avansează. Răspunsul se construiește **bit cu bit**, ca scrierea în baza 2
- Fără mid și fără intervale; același $O(\log V)$ teste. Folosit la strămoșii dintr-un arbore (LCA, al k-lea strămoș) și la căutarea în arbori Fenwick (al k-lea element)

```
long long x = st - 1;                    // inaintea domeniului
for (int p = 60; p >= 0; p--)            // 2^p nu depaseste dr - st
    if (x + (1LL << p) <= dr && check(x + (1LL << p)))
        x += 1LL << p;
// x = cea mai mare valoare cu check adevarat (st - 1 daca nu exista)
```

Căutare exponențială

Când limita superioară nu se cunoaște, se testează $1, 2, 4, 8, \ldots$ cât timp $\text{check}(2^p)$ este adevărat; la primul test fals, răspunsul se află în $[2^{p-1}, 2^p)$ și se face căutare binară pe acest interval. Sunt $O(\log r)$ teste, unde r este răspunsul: eficientă și când răspunsul este mic într-un domeniu uriaș.

Căutare ternară și căutare binară în paralel

Căutare ternară

- Pentru o funcție **unimodală** (crește strict, apoi scade strict) se caută maximul: din $m_1 < m_2$ se elimină o treime; dacă $f(m_1) < f(m_2)$, maximul nu se află în $[l, m_1]$
- Zonele constante strică metoda; pe întregi merge căutarea binară pe $f(m) < f(m+1)$; secțiunea de aur face mai puține evaluări

```
double l = st, r = dr;
for (int it = 0; it < 100; it++) {
    double m1 = l + (r - l) / 3, m2 = r - (r - l) / 3;
    if (f(m1) < f(m2)) l = m1; else r = m2;   // pentru maxim
}
```

Căutare binară în paralel

- Q întrebări, fiecare cu propria căutare binară pe „timp” (de exemplu: după câte muchii adăugate sunt u și v conectate), cu o structură incrementală (DSU, Fenwick)
- Căutările rulează simultan: în fiecare rundă întrebările se grupează după mid, timpul se parcurge **o singură dată**, iar la momentul mid se răspunde la teste; după $\lceil \log T \rceil$ runde totul e rezolvat (clasic: Meteors, POI)

Cuprins

1. Monotonia
2. Two pointers
3. Căutare binară pe rezultat
4. Alte variante
5. Recapitulare și probleme propuse

Recapitulare

	Two pointers	**Căutare binară pe rezultat**
Se caută	secvențe, perechi, ferestre	o valoare: cel mai mic / mare x
Monotonie	$r(l)$ nu scade când l crește	$\mathrm{check}(x)$ trece o singură dată de la fals la adevărat
Se menține	suma, frecvențele, numărul de valori distincte din fereastră	nimic: check se reface la fiecare test
Complexitate	$O(n)$ amortizat	$O(\log V \cdot \mathrm{cost}(\mathrm{check}))$
Cuvinte-cheie	„cea mai lungă / scurtă secvență", „câte secvențe", perechi cu o sumă dată	„cel mai mic / mare x astfel încât", „minimizează maximul", „maximizează minimul"

Alegerea tehnicii

1. Se caută **ce este monoton**: ce proprietate trece o singură dată de la fals la adevărat?
2. Dacă variabila este **capătul unei secvențe** și proprietatea se actualizează ușor la mutarea unui capăt $\Rightarrow$ **two pointers**
3. Dacă variabila este **valoarea răspunsului** și se poate verifica ușor o valoare dată $\Rightarrow$ **căutare binară pe rezultat**
4. Uneori cele două se combină: căutare binară pe valoare, cu two pointers în verificare (de exemplu K-th Sum, la problemele propuse)
5. Pentru funcții unimodale $\Rightarrow$ căutare ternară; pentru multe întrebări cu verificare costisitoare $\Rightarrow$ căutare binară în paralel

Înainte de a scrie cod

Se verifică monotonia pe un exemplu mic și se stabilesc limitele, tipul de date și complexitatea verificării.

Probleme propuse: ușoare

Problemă	Sursă	Tehnică
buldo (în lecție)	(Kilonova, OJI 2020 cl. IX)	căutare binară
Palindrom (în lecție)	(Kilonova, OJI 2023 cl. VII)	two pointers
pietricele (în lecție)	(Kilonova, ONI 2023 cl. IX)	căutare binară
criptografie (în lecție)	(Kilonova, ONI 2019 cl. VIII)	two pointers
amat	(Kilonova, ONI 2019 cl. IX)	căutare binară
Jumping Through Segments	(Codeforces 1907D)	căutare binară
Eating Candies	(Codeforces 1669F)	two pointers
Robin Hood in Town	(Codeforces 2014C)	căutare binară

Probleme propuse: medii

Mapat	(Kilonova, ONI 2026 cl. VII)
kmajo	(Kilonova, ONI 2024 cl. VIII)
intergalactic	(Kilonova, ONI 2021 Baraj Juniori)
fibopower	(Kilonova, ONI 2026 Baraj Juniori)
bossfight	(Kilonova, Lot 2023 Baraj 2 Juniori)
zapada	(Kilonova, ONI 2026 cl. VIII)
geologie	(Kilonova, ONI 2024 cl. VIII)
Subarray Distinct Values	(CSES 2428)
Array Division	(CSES 1085)
Final Boss	(Codeforces 1985F)
Tea Tasting	(Codeforces 1795C)

Probleme propuse: grele

Clone	(Kilonova, Stelele Informaticii 2026)
Choose Interval	(Kilonova, RMI 2024)
Pinball	(Kilonova, RMI 2023)
Eras	(Kilonova, ONI 2024 cl. IX)
tactica	(Kilonova, Lot 2026 Baraj 3 Juniori)
f1	(Kilonova, Lot 2023 Baraj 1 Juniori)
Max Median	(Codeforces 1486D)
MEX vs MED	(Codeforces 1744F)
Cleaning the Phone	(Codeforces 1475D)
K-th Sum	(Codeforces EDU 6/5)

Resurse și bibliografie

1. **USACO Guide** — Binary Search (EN)
2. **USACO Guide** — Two Pointers (EN)
3. **cp-algorithms.com** — Binary Search, inclusiv căutarea cu puteri ale lui 2 și Parallel Binary Search (EN)
4. **cp-algorithms.com** — Ternary Search (EN)
5. **Codeforces EDU** — Binary Search, pașii 1–5 (EN)
6. **Codeforces EDU** — Two Pointers Method (EN)
7. **Codeforces blog** — Parallel Binary Search [tutorial], himanshujaju (EN)
8. **Codeforces blog** — Binary Search on the Answer: A Simple Trick That Wins Many Problems (EN)
9. **Competitive Programmer's Handbook** — A. Laaksonen, capitolul despre sortare și căutare binară (EN)

▷ *Pentru „Știați că?": J. Bloch, „Extra, Extra — Read All About It: Nearly All Binary Searches and Mergesorts are Broken" (2006).*